

Sistema de Gestión Hotelera

Documento Técnico - Segunda Entrega

Implementación de Repositorios, Servicios y Controladores CRUD

Luis Miguel Viuche - 20212020082

María Fernanda Vélez Benítez - 20232020104

Sharon Stefany Castilla Araque - 20251020089

David Santiago Ospina Carrillo - 20232020013

Curso: Programación Avanzada con Java y Spring Boot

Grupo ACM

2026

Repositorio: <https://github.com/TheMiniWolf/Grupo-de-trabajo-ACM>

Índice

1. Introducción	2
2. Objetivo General	2
3. Objetivos Específicos	2
4. Arquitectura Implementada	3
5. Cambios Respecto a la Primera Entrega	3
6. Enumeraciones	4
7. Repositorios	5
8. Servicios	6
9. Controladores REST	8
10. Manejo Global de Excepciones	9
11. Migraciones de Base de Datos (Flyway)	10
12. Entidades JPA - Ajustes de Segunda Entrega	11
13. Pruebas Implementadas	13
14. Estructura del Proyecto	13
15. Dependencias del Proyecto	14
16. Configuración de Persistencia	14
17. Conclusiones	15

1. Introducción

El Sistema de Gestión Hotelera es una solución de software backend desarrollada con Spring Boot y PostgreSQL, orientada a la administración integral de establecimientos hoteleros. Esta segunda entrega continúa directamente sobre la estructura base establecida en la primera fase del proyecto.

En la entrega anterior se realizó el levantamiento de requerimientos, el diseño del modelo relacional con 12 entidades normalizadas, la implementación de entidades JPA y la configuración de persistencia con Flyway. En esta segunda fase se construyó la capa funcional del backend mediante la implementación de repositorios JPA, servicios con lógica de negocio y controladores REST con operaciones CRUD completas, siguiendo el patrón de arquitectura en capas utilizado en aplicaciones empresariales con Spring Boot.

2. Objetivo General

Implementar la capa de acceso a datos, servicios y controladores REST del Sistema de Gestión Hotelera utilizando Spring Boot, aplicando operaciones CRUD mediante interfaces JpaRepository y exponiendo una API REST organizada por entidad.

3. Objetivos Específicos

- Implementar repositorios JPA con consultas derivadas para cada entidad principal.
- Crear servicios que centralicen la lógica de negocio y validen las operaciones CRUD.
- Desarrollar controladores REST para exponer los endpoints del sistema.
- Aplicar arquitectura por capas separando entidad, repositorio, servicio y controlador.
- Mantener compatibilidad con PostgreSQL, JPA/Hibernate y las migraciones Flyway existentes.
- Preparar la estructura para futuras funcionalidades avanzadas como seguridad JWT.

4. Arquitectura Implementada

El proyecto sigue una arquitectura en cuatro capas bien definidas. Las capas de la primera entrega permanecen intactas; la segunda entrega agrega las tres capas superiores, como muestra el Cuadro 1.

Cuadro 1: Capas de la arquitectura del sistema

Capa	Paquete	Responsabilidad	Estado
Entidad (Entity)	entidad/	Mapeo objeto-relacional con JPA/Hibernate	Primera entrega — sin cambios
Repositorio (Repository)	repositorio/	Acceso a datos vía JpaRepository	Segunda entrega — nuevo
Servicio (Service)	servicio/	Lógica de negocio y validaciones	Segunda entrega — nuevo
Controlador (Controller)	controlador/	Endpoints REST HTTP	Segunda entrega — nuevo

El flujo completo de una petición HTTP atraviesa las cuatro capas en orden descendente:

```
Cliente HTTP → Controller → Service → Repository → PostgreSQL
               ←           ←           ←           ←
```

5. Cambios Respecto a la Primera Entrega

5.1. Renombrado de la entidad Hotel

La entidad que en el diseño conceptual se denominaba Hotel pasó a llamarse CadenaHotel en la implementación. Este cambio refleja con mayor precisión el modelo de datos adoptado, donde una cadena hotelera es el nivel corporativo superior que agrupa múltiples sucursales físicas. El Cuadro 2 resume los cambios de nomenclatura.

Cuadro 2: Cambios de nomenclatura — Hotel → CadenaHotel

Concepto	Primera entrega (diseño)	Segunda entrega (implementación)
Clase Java	Hotel	CadenaHotel
Tabla SQL	hotel	cadena_hotel
Endpoint REST	/api/hoteles	/api/cadenas-hotel
Servicio	HotelService	CadenaHotelService
Repositorio	HotelRepository	CadenaHotelRepository

5.2. Nuevos paquetes creados

Se agregaron tres paquetes nuevos al proyecto sin modificar ningún archivo existente de la primera entrega:

- repositorio/ — 12 interfaces que extienden JpaRepository.
- servicio/ — 12 clases @Service con lógica de negocio.
- controlador/ — 12 clases @RestController con endpoints REST.

5.3. Paquetes de soporte adicionales

Se crearon dos paquetes de soporte no contemplados en la primera entrega:

- enums/ — 6 enumeraciones Java para tipos y estados controlados, alineadas con los CHECK de la base de datos.
- excepcion/ — Manejo global centralizado de errores HTTP mediante @RestControllerAdvice.

5.4. Ajustes en entidades JPA

Las entidades de la primera entrega se extendieron con el patrón @JsonIgnore / @JsonProperty para controlar la serialización JSON de las relaciones lazy. Esto permite que el cliente REST trabaje con identificadores planos (ej. reservaId) en lugar de objetos anidados, evitando problemas de inicialización lazy y ciclos de serialización.

5.5. Migraciones Flyway

Las migraciones originales (V1, V2, V3) no se modificaron. La segunda entrega las reutiliza tal como fueron definidas; Flyway detecta que ya están aplicadas y no las vuelve a ejecutar.

6. Enumeraciones

Se crearon 6 enums en el paquete enums/ para reemplazar cadenas libres por dominios controlados, garantizando coherencia entre la validación Java y las restricciones CHECK de PostgreSQL. El Cuadro 3 los describe.

Cuadro 3: Enumeraciones del dominio

Enum	Valores	Usado en
EstadoGeneral	ACTIVO, INACTIVO	CadenaHotel, Sucursal
EstadoHabitacion	DISPONIBLE, OCUPADA, MANTENIMIENTO, LIMPIEZA	Habitacion
EstadoPago	PENDIENTE, COMPLETADO, FALLIDO, REEMBOLSADO	Pago
EstadoReserva	PENDIENTE, CONFIRMADA, CANCELADA, FINALIZADA	Reserva
MetodoPago	EFFECTIVO, TARJETA, TRANSFERENCIA, PAGO_DIGITAL	Pago
TipoServicio	RESTAURANTE, SPA, LAVANDERIA, TRANSPORTE	Servicio

7. Repositorios

Cada repositorio extiende JpaRepository<Entidad, Long>, lo que provee automáticamente las operaciones save(), findById(), findAll() y deleteById(). Adicionalmente se definieron consultas derivadas para cubrir necesidades específicas del dominio. El Cuadro 4 lista las consultas de cada repositorio.

Cuadro 4: Repositorios y sus consultas derivadas

Repositorio	Consultas derivadas
UsuarioRepository	findByEmail, existsByEmail, existsByDocumento, findByRol_Id
RolRepository	findByNombre, existsByNombre
CadenaHotelRepository	findByEstado
SucursalRepository	findByCadenaHotelId, findByCiudad
HabitacionRepository	findBySucursal_Id, findByEstado, findByDisponibleTrue, existsBySucursal_IdAndNumero
ReservaRepository	findByCliente_Id, findByEstado
ReservaHabitacionRepository	findByReserva_Id, findByHabitacion_Id
PagoRepository	findByReserva_Id, existsByReserva_Id, findByEstado
FacturaRepository	findByPago_Id, findByNumeroFactura, existsByNumeroFactura, existsByPago_Id
ReservaServicioRepository	findByReserva_Id, findByServicio_Id
ServicioRepository	findByActivoTrue, findByTipo
TipoHabitacionRepository	existsByNombre

Nota técnica: en los repositorios con relaciones FK mapeadas con `@JsonProperty` se usa la notación con guion bajo (ej. `findByReserva_Id`) para forzar la travesía por el objeto JPA y evitar colisiones con los getters de serialización JSON, tal como se muestra en el Listing 1.

```
@Repository
public interface PagoRepository extends JpaRepository<Pago, Long> {

    // Guion bajo: fuerza la travesía pago → reserva.id
    // (evita colisión con el getter getReservaId())
    Optional<Pago> findByReserva_Id(Long reservaId);

    boolean existsByReserva_Id(Long reservaId);

    List<Pago> findByEstado(EstadoPago estado);
}
```

Listing 1: Ejemplo: `PagoRepository.java` con consultas derivadas

8. Servicios

La capa de servicios centraliza la lógica de negocio. Todos los servicios están anotados con `@Service` y `@Transactional`. Las operaciones de solo lectura usan `@Transactional(readOnly = true)` para optimizar el rendimiento en PostgreSQL.

8.1. Patrón de inyección

Todos los servicios usan inyección por constructor, lo que facilita las pruebas unitarias y cumple con las buenas prácticas de Spring Boot 4.0.6:

```
@Service
@Transactional
public class HabitacionService {

    private final HabitacionRepository habitacionRepository;
    private final SucursalRepository sucursalRepository;
    private final TipoHabitacionRepository tipoHabitacionRepository;

    public HabitacionService(HabitacionRepository hr,
                             SucursalRepository sr,
                             TipoHabitacionRepository tr) {
        this.habitacionRepository = hr;
        this.sucursalRepository = sr;
        this.tipoHabitacionRepository = tr;
    }
}
```

Listing 2: Patrón de inyección por constructor — `HabitacionService.java`

8.2. Validaciones de negocio por servicio

Cada servicio implementa validaciones específicas de su dominio antes de persistir o modificar datos. El Cuadro 5 resume las validaciones de cada servicio.

Cuadro 5: Validaciones de negocio por servicio

Servicio	Validaciones implementadas
UsuarioService	Email único, documento único, resolución de Rol por FK.
RolService	Nombre de rol único al crear y al actualizar.
CadenaHotelService	Estado por defecto ACTIVO al crear.
SucursalService	Resolución de CadenaHotel por FK; estado ACTIVO por defecto.
HabitacionService	Número único por sucursal; resolución de Sucursal y TipoHabitacion por FK; estado y disponibilidad por defecto.
ReservaService	fecha_salida > fecha_entrada; total >= 0; resolución de Usuario por FK; estado PENDIENTE por defecto.
PagoService	Una reserva solo puede tener un pago; fecha por defecto NOW(); estado PENDIENTE por defecto.
FacturaService	Número de factura único; un pago solo puede tener una factura; cálculo automático del total (subtotal + impuestos).
ReservaServicioService	Cantidad >= 1; precio unitario tomado del Servicio si no se envía; cálculo automático del subtotal.
ReservaHabitacionService	Resolución de Reserva y Habitacion por FK.
ServicioService	activo = true por defecto al crear.
TipoHabitacionService	Nombre de tipo de habitación único.

8.3. Ejemplo de servicio

El Listing 3 muestra la implementación del servicio de reservas, incluyendo las validaciones de fechas y total, y el estado por defecto al crear:

```
@Service
@Transactional
public class ReservaService {

    public Reserva crear(Reserva reserva) {
        reserva.setCliente(resolverCliente(reserva.getClienteId()));
        validarFechas(reserva);
        validarTotal(reserva);
        if (reserva.getEstado() == null)
            reserva.setEstado(EstadoReserva.PENDIENTE);
        reserva.setId(null);
        return reservaRepository.save(reserva);
    }

    private void validarFechas(Reserva reserva) {
        if (!reserva.getFechaSalida().isAfter(reserva.getFechaEntrada()))
            throw new ReglaNegocioException(
                "La fecha de salida debe ser posterior a la de entrada.");
    }
}
```

Listing 3: Fragmento de ReservaService.java con validaciones de negocio

9. Controladores REST

Se implementaron 12 controladores REST, uno por cada entidad principal. Todos siguen el mismo patrón estructural: cinco endpoints que cubren las operaciones CRUD. El Cuadro 6 lista los controladores y sus URL base.

Cuadro 6: Controladores REST y URLs base

Controlador	URL base	Entidad gestionada
RolController	/api/roles	Rol
UsuarioController	/api/usuarios	Usuario
CadenaHotelController	/api/cadenas-hotel	CadenaHotel
SucursalController	/api/sucursales	Sucursal
TipoHabitacionController	/api/tipos-habitacion	TipoHabitacion
HabitacionController	/api/habitaciones	Habitacion
ReservaController	/api/reservas	Reserva
ReservaHabitacionController	/api/reserva-habitaciones	ReservaHabitacion
PagoController	/api/pagos	Pago
FacturaController	/api/facturas	Factura
ServicioController	/api/servicios	Servicio
ReservaServicioController	/api/reserva-servicios	ReservaServicio

El Cuadro 7 describe las cinco operaciones disponibles en cada controlador:

Cuadro 7: Operaciones CRUD por controlador

Método HTTP	Ruta	Acción	Código de respuesta
GET	/api/{recurso}	Listar todos los registros	200 OK
GET	/api/{recurso}/{id}	Obtener un registro por ID	200 OK / 404 Not Found
POST	/api/{recurso}	Crear un nuevo registro	201 Created
PUT	/api/{recurso}/{id}	Actualizar un registro existente	200 OK / 404 Not Found
DELETE	/api/{recurso}/{id}	Eliminar un registro	204 No Content / 404 Not Found

El Listing 4 muestra la estructura típica de un controlador REST en el sistema:

```
@RestController
@RequestMapping("/api/reservas")
public class ReservaController {

    private final ReservaService reservaService;

    public ReservaController(ReservaService reservaService) {
        this.reservaService = reservaService;
    }

    @GetMapping
    public List<Reserva> listar() {
        return reservaService.listar();
    }
}
```



```
@GetMapping("/{id}")
public Reserva obtener(@PathVariable Long id) {
    return reservaService.obtener(id);
}

@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public Reserva crear(@RequestBody Reserva reserva) {
    return reservaService.crear(reserva);
}

@PutMapping("/{id}")
public Reserva actualizar(@PathVariable Long id,
                        @RequestBody Reserva reserva) {
    return reservaService.actualizar(id, reserva);
}

@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void eliminar(@PathVariable Long id) {
    reservaService.eliminar(id);
}
```

Listing 4: Estructura típica de un controlador REST — ReservaController.java

10. Manejo Global de Excepciones

Se implementó un paquete `excepcion/` con manejo centralizado de errores mediante `@RestControllerAdvice`. Esto garantiza respuestas JSON uniformes para todos los errores de la API, independientemente de qué controlador los origine. El Cuadro 8 describe las excepciones implementadas.

Cuadro 8: Excepciones y sus respuestas HTTP

Excepción	Código HTTP	Cuándo se lanza
RecursoNoEncontradoException	404 Not Found	<code>findById()</code> no encuentra el registro solicitado.
ReglaNegocioException	400 Bad Request	Validaciones de unicidad, fechas incoherentes o montos inválidos.
DataIntegrityViolationException	409 Conflict	Violación de restricciones de clave foránea o unicidad en la base de datos.

Todas las respuestas de error siguen la misma estructura JSON, tal como se muestra en el Listing 5:

```
{
  "estado": 404,
  "error": "Not Found",
  "mensaje": "Usuario con id 5 no encontrado.",
  "fecha": "2026-05-15T10:23:45"
}
```

Listing 5: Estructura JSON uniforme de respuesta de error

11. Migraciones de Base de Datos (Flyway)

Las migraciones se ejecutan automáticamente al iniciar la aplicación. Las tres versiones definidas en la primera entrega permanecen sin cambios; Flyway detecta que ya están aplicadas (registro en la tabla `flyway_schema_history`) y las omite en los arranques posteriores.

11.1. V1__crear_tablas.sql

Crea las 12 tablas del sistema con sus columnas, tipos de dato y claves foráneas. Se listan a continuación las tablas incluidas:

- `cadena_hotel` — Cadena hotelera.
- `sucursal` — Sede física de una cadena.
- `rol y usuario` — Control de acceso.
- `tipo_habitacion y habitacion` — Catálogo y unidades de alojamiento.
- `reserva y reserva_habitacion` — Ciclo de vida de una reserva.
- `servicio y reserva_servicio` — Servicios adicionales.
- `pago y factura` — Transacciones y documentos fiscales.

11.2. V2__restricciones_e_indices.sql

Agrega tres grupos de restricciones complementarias a las tablas ya creadas:

- Unicidad: email y documento en usuario; nombre en rol; número en factura; número por sucursal en habitacion.
- Dominios controlados (CHECK): alineados con los valores de los enums Java para estado, metodo_pago y tipo.
- Reglas de negocio: `fecha_salida > fecha_entrada` en reserva; `monto > 0` en pago; `precio >= 0` en habitacion y servicio.
- Índices en claves foráneas para optimizar consultas con JOIN.

11.3. V3__datos_iniciales.sql

Inserta los valores fijos de catálogo que no cambian en producción, tal como se muestra en el Listing 6:

```
insert into rol (nombre, descripcion) values
  ('CLIENTE',      'Usuario que realiza y gestiona sus reservas'),
  ('EMPLEADO',     'Personal operativo de una sucursal'),
  ('ADMINISTRADOR', 'Gestiona hoteles, usuarios y el sistema');

insert into tipo_habitacion (nombre, descripcion) values
  ('SENCILLA',     'Habitacion individual basica'),
  ('DOBLE',        'Habitacion para dos personas'),
  ('SUITE',        'Suite con sala independiente'),
  ('PRESIDENCIAL', 'Suite presidencial de lujo');
```

Listing 6: V3__datos_iniciales.sql — Datos de catálogo

12. Entidades JPA - Ajustes de Segunda Entrega

Las entidades JPA definidas en la primera entrega se extendieron con el patrón `@JsonIgnore` / `@JsonProperty` para controlar la serialización JSON de las relaciones lazy. Cada entidad está anotada con `@Entity`, `@Table`, y usa Lombok (`@Data`, `@NoArgsConstructor`, `@AllArgsConstructor`) para reducir código repetitivo.

12.1. Patrón @JsonIgnore / @JsonProperty

El Listing 7 muestra cómo se aplica el patrón en la entidad Usuario para exponer la FK como un id plano en el JSON sin serializar el objeto Rol completo:

```
@Entity @Table(name = "usuario")
@Data @NoArgsConstructor @AllArgsConstructor
public class Usuario {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @JsonIgnore // no aparece en el JSON
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "rol_id", nullable = false)
    private Rol rol;

    @Column(nullable = false) private String nombre;
    @Column(nullable = false) private String apellido;
    @Column(nullable = false) private String email;

    @JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
    @Column(nullable = false) private String password;

    @Column(nullable = false) private String telefono;
    @Column(nullable = false) private String documento;

    @CreationTimestamp
    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @JsonProperty("rolId") // expone solo el id de la FK
    public Long getRolId() {
        return rol != null ? rol.getId() : null;
    }
}
```

Listing 7: Entidad Usuario.java con patrón @JsonIgnore / @JsonProperty

12.2. Entidad Reserva

```
@Entity @Table(name = "reserva")
@Data @NoArgsConstructor @AllArgsConstructor
public class Reserva {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @JsonIgnore
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "cliente_id", nullable = false)
    private Usuario cliente;

    @Column(name = "fecha_entrada", nullable = false)
    private LocalDate fechaEntrada;
    @Column(name = "fecha_salida", nullable = false)
    private LocalDate fechaSalida;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 30)
    private EstadoReserva estado;

    @Column(nullable = false) private BigDecimal total;
```

```

@CreationTimestamp
@Column(name = "created_at", nullable = false, updatable = false)
private LocalDateTime createdAt;

@JsonProperty("clienteId")
public Long getClienteId() {
    return cliente != null ? cliente.getId() : null;
}
}

```

Listing 8: Entidad Reserva.java

12.3. Entidades Pago y Factura

Pago y Factura usan `@OneToOne` con `unique = true` para implementar la relación 1:1 con Reserva y Pago respectivamente, garantizando que cada reserva tenga a lo sumo un pago y cada pago a lo sumo una factura.

```

@Entity @Table(name = "pago")
@Data @NoArgsConstructor @AllArgsConstructor
public class Pago {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @JsonIgnore
    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "reserva_id", nullable = false, unique = true)
    private Reserva reserva;

    @Column(nullable = false) private BigDecimal monto;

    @Enumerated(EnumType.STRING)
    @Column(name = "metodo_pago", nullable = false, length = 30)
    private MetodoPago metodoPago;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 30)
    private EstadoPago estado;

    @Column(name = "fecha_pago", nullable = false)
    private LocalDateTime fechaPago;
}

```

Listing 9: Entidad Pago.java

13. Pruebas Implementadas

Se implementaron cuatro clases de prueba en el paquete `test/` para validar el correcto funcionamiento de las capas implementadas, usando H2 en memoria como base de datos de prueba. El Cuadro 9 las describe.

Cuadro 9: Clases de prueba implementadas

Clase de prueba	Tipo	Qué valida
ProyectoHotelApplicationTests	Integración	El contexto de Spring carga correctamente con todas las capas.

Clase de prueba	Tipo	Qué valida
RolControllerTest	Integración (MockMvc)	Endpoints CRUD del controlador de Rol, incluyendo validaciones de unicidad.
UsuarioControllerTest	Integración (MockMvc)	Endpoints CRUD del controlador de Usuario con resolución de FK a Rol.
ConsultasDerivadasFkTest	Integración (JPA)	Consultas derivadas con FK en repositorios usando notación de guion bajo.

14. Estructura del Proyecto

```

ProyectoHotel/
├── src/
│   ├── main/
│   │   ├── java/com/acm/proyectohotel/
│   │   │   ├── entidad/           # 12 entidades JPA
│   │   │   ├── enums/           # 6 enumeraciones de dominio
│   │   │   ├── repositorio/      # 12 interfaces JpaRepository
│   │   │   ├── servicio/        # 12 clases @Service
│   │   │   ├── controlador/     # 12 clases @RestController
│   │   │   ├── excepcion/       # Manejo global de errores
│   │   │   └── ProyectoHotelApplication.java
│   │   └── resources/
│   │       ├── db/migration/     # V1, V2, V3 (Flyway)
│   │       ├── application.properties
│   │       └── application-test.properties
│   └── test/                    # 4 clases de prueba
├── docs/                       # Documentación del proyecto
├── pom.xml
└── README.md
  
```

Listing 10: Estructura de directorios del proyecto

15. Dependencias del Proyecto

Las siguientes dependencias conforman el pom.xml del proyecto, bajo Spring Boot 4.0.6 y Java 17. Son las mismas de la primera entrega más spring-boot-starter-web para habilitar los controladores REST:

```

<!-- Spring Data JPA + Hibernate -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- Spring Web MVC (REST Controllers) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<!-- Flyway (migraciones automaticas) -->
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-database-postgresql</artifactId>
  
```

```
</dependency>

<!-- Driver PostgreSQL -->
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>

<!-- Lombok -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <scope>provided</scope>
</dependency>

<!-- H2 para pruebas -->
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>test</scope>
</dependency>
```

Listing 11: Dependencias en pom.xml

16. Configuración de Persistencia

```
spring.application.name=ProyectoHotel

spring.datasource.url=${URL_BD:jdbc:postgresql://localhost:5432/proyecto_hotel}
spring.datasource.username=${USUARIO_BD:postgres}
spring.datasource.password=${CLAVE_BD:postgres}

# Flyway es el dueño del esquema; Hibernate solo valida.
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.open-in-view=false
spring.jpa.show-sql=true

spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration
```

Listing 12: application.properties

17. Conclusiones

En esta segunda entrega se completó satisfactoriamente la capa funcional del backend del Sistema de Gestión Hotelera. Los logros principales fueron:

- Se implementaron 12 repositorios con consultas derivadas que cubren todos los casos de uso de acceso a datos, incluyendo búsquedas por estado, por FK y verificaciones de existencia.
- Se crearon 12 servicios con validaciones de negocio robustas: unicidad de campos, coherencia de fechas, cálculo automático de montos y resolución de FK con mensajes de error descriptivos.
- Se expusieron 60 endpoints REST (5 operaciones por entidad) organizados de forma consistente y documentados con códigos HTTP apropiados.
- Se incorporó un sistema de manejo global de errores que garantiza respuestas JSON uniformes en toda la API, incluyendo errores de integridad referencial de PostgreSQL.

- El proyecto queda preparado para una tercera entrega donde se pueden agregar autenticación JWT, DTOs con Bean Validation, paginación y endpoints de consulta avanzados.